

M03 / ASSESSMENT

M03 Module Revision Test

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first	3
Rules	3
T01 - DE03-01 INNER JOIN and LEFT JOIN	3
T02 - DE03-02 Cardinality, row multiplication and join validation	3
T03 - DE03-03 UNION and UNION ALL	3
T04 - DE03-04 String functions for cleaning and standardization	3
T05 - DE03-05 Dates, timestamps and time analysis	4
T06 - DE03-06 Subqueries and nested logic	4
T07 - DE03-07 CTEs and readable multi-step SQL	4
T08 - DE03-08 Window functions and PARTITION BY	4
T09 - DE03-09 ROW_NUMBER, ranking and LAG/LEAD	4
T10 - DE03-10 Advanced aggregation and multi-grain reasoning	4
T11 - DE03-11 PostgreSQL setup, schemas and dialect awareness	5
T12 - DE03-12 Constraints and transactions	5
T13 - DE03-13 Indexes and EXPLAIN basics	5
T14 - DE03-14 Views and reusable database logic	5
T15 - DE03-15 Integrated unseen SQL challenge	5
T16 - DE03-16 Timed SQL interview set and explanation	6

Read this first

MODULE REVISION TEST

Save your answers before opening the protected answer guide. No forced timer. First prove the competency untimed; later you may record interview-style time if useful. This test does not replace a protected Gate.

Coverage: 16 canonical lessons

Route: learner book -> practice -> revision -> closed-book test -> protected answers -> changed retry.

Rules

Work with learner book, review book, answer guide and AI solving closed. Save one answer/evidence item for every lesson. If a task needs a PC, write your prediction first, then run it. Mark PASS only when the result and your explanation agree.

T01 - DE03-01 | INNER JOIN and LEFT JOIN

Return every order but attach a customer name only when the customer belongs to North. Keep all original orders, including those outside North and O106. Explain why WHERE region = 'North' would fail that preservation rule.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T02 - DE03-02 | Cardinality, row multiplication and join validation

Count all rows from a LEFT JOIN of orders to items. Also count non-null item line numbers and distinct order IDs. Explain why these three counts differ, and identify the row causing the difference between the first two counts.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T03 - DE03-03 | UNION and UNION ALL

Find categories appearing in course.retry_sales on both November 1 and November 2 using INTERSECT. Then find categories from November 1 absent on November 2 using EXCEPT.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T04 - DE03-04 | String functions for cleaning and standardization

Using only the three literal inputs below, calculate raw value, cleaned value and cleaned length: ' Bag ', 'BAG' and ' '. You can use UNION ALL to create three SELECT statements. Decide whether a blank cleaned value should have length zero or a null length under the chosen missing-label rule.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T05 - DE03-05 | Dates, timestamps and time analysis

Select September 1 according to UTC instead of India. Then explain why a different pair of IDs is correct despite the stored events being unchanged.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T06 - DE03-06 | Subqueries and nested logic

Find customers with at least two orders using a scalar correlated COUNT. Return ID and name, one row per customer. Explain why the count cannot be replaced by EXISTS alone.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T07 - DE03-07 | CTEs and readable multi-step SQL

Use item_totals to identify order headers with no matching item summary. Return their order IDs and explain why you test the right-side key rather than item_gross = 0.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T08 - DE03-08 | Window functions and PARTITION BY

Produce a running gross total for course.retry_sales by ordered_on, breaking ties with order_id. Use an explicit ROWS frame and predict the total after R202.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T09 - DE03-09 | ROW_NUMBER, ranking and LAG/LEAD

For course.retry_sales in date/ID order, return each amount, its previous amount and the difference from the previous amount. Preserve the first difference as NULL. Explain why filling that first difference with zero would be an added assumption.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T10 - DE03-10 | Advanced aggregation and multi-grain reasoning

Using the same item-to-order preparation, report the count and average gross for non-empty orders only. Independently calculate total gross divided by that order count. Explain the difference from the all-order average in the guided report.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T11 - DE03-11 | PostgreSQL setup, schemas and dialect awareness

Create a temporary table called `desk_retry` with text reference as primary key and a non-null integer units column. Insert X01 with units 3, then read it. Explain whether merely saving those statements to a file creates the table.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T12 - DE03-12 | Constraints and transactions

In a new temporary stock table, insert ONE/5. Begin a transaction, reduce units to 4 and COMMIT. Verify 4 remains. Attempt a second insertion with the same key and record the error without removing the original record. Explain why ROLLBACK after the successful commit cannot restore the earlier 5.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T13 - DE03-13 | Indexes and EXPLAIN basics

Inspect a query selecting `customer_no = 21`, for which no `customer_no` index has been created. Predict the number of matching rows, execute a COUNT to check it, and inspect the plan. Explain why an index on `order_no` does not automatically become an index on `customer_no`.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T14 - DE03-14 | Views and reusable database logic

Using the disposable `view_source` table and `view_demo`, begin a transaction, change units from 3 to 4, inspect the view, then roll back and inspect it again. Explain why the view follows the transaction's visible source state.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T15 - DE03-15 | Integrated unseen SQL challenge

Use `course.challenge_retry_customers`, `course.challenge_retry_orders` and `course.challenge_retry_items`. Build all-order totals and region totals from a blank editor. Keep empty orders and unknown identity, reconcile counts and amounts, and calculate the average gross per order over the complete order population. Explain why averaging region averages equally would be unsafe. Do not reuse the previous result numbers.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY

T16 - DE03-16 | Timed SQL interview set and explanation

Use `course.practice_sales` for two fresh interview questions: choose the highest-gross sale per category, resolving ties by ascending ID; then return categories whose known-paid gross exceeds 50. Explain when `WHERE` happens relative to `GROUP BY` and a window calculation. For a third task, use `course.challenge_retry` tables to prove the all-order total and state its average denominator. Save the time and assistance record separately from the first set.

Save: answer / code / command / result + one validation note. Status: PASS / REVIEW NEEDED / RETRY